

Tabular RL Project

Due March 7th, 2026

General Instructions

For the assigned project, each team member must implement one distinct algorithm (ensuring no overlap with teammates) from the following five options to find the optimal policy. All implementations must be written from scratch in Python, using strictly numpy or torch:

1. Value iteration
2. Policy iteration
3. MC control (on and/or off)
4. TD control using SARSA(λ)
5. Q learning

For the first and second algorithm, assume transition probabilities and rewards are known and then show that the algorithms converge to the optimal value function/policy. For rest of the algorithms, use the transition probabilities to only generate the episodes online. For algorithms 3-5, plot the total discounted reward per episode and cumulative rewards over episodes. Also plot the instantaneous and cumulative regrets for the RL algorithms defined as follows.

Instantaneous episodic regret: It is the difference between the total discounted reward earned by your RL algorithm in the current episode and the expected cumulative discounted reward earned by the optimal policy in an episode (essentially $V_\alpha(s)$ if the episode started in state s). It measures how much reward you are losing choosing a sub-optimal policy.

Cumulative regret: At the current iteration/episode, this is running/cumulative sum of all the previous instantaneous episodic regrets till now.

Value and Policy iteration for the finite time horizon: Recall from the class that VI and PI were used for infinite horizon discounted problem settings. In fact, in such settings deterministic, stationary and Markovian policies are optimal and so in policy iteration we always chose deterministic and stationary policy. We also know that for a finite horizon problem, the optimal policy is non-stationary. In that case, how will you adapt that equation to solve the finite time horizon problem? Give it a thought! The following paragraph with Q learning for finite horizon will act as a hint.

Q-learning for finite horizon: Q-learning in the standard form assumes deterministic and stationary policy for an infinite horizon MDP. In many of the projects, the problem only makes sense for finite horizons. You can adapt Q-learning to finite horizon as shown in this link. Instead of assuming stationary policy and using $Q(s, a)$, use the notation $Q_t(s, a)$, and make the update to it in a similar fashion. See the following algorithm from the above paper.

Project 1: Inventory Management problem

Retail stores stockpile products in warehouses to meet the random demand. Additional stocks are procured at regular intervals. Let X_t denote the amount of stock before the t -th procurement. In this example, time denotes the number of additional stock procurements. At time t , the store may procure an additional stock U_t ($\leq U$) units for price p per unit. Thus the total procurement cost is pU_t .

The random demand W_t is i.i.d. with distribution P_W . The stock available at the next time is $X_{t+1} = X_t + U_t - W_t$, where a negative stock denotes backlogged demand.

Algorithm 1 Finite Horizon Q-Learning

Notation:

$Q_n^m(i, a)$: Q-value at state i , action a , stage n , recursion m .
 $a(m)$: step-size at recursion index m
 $Q_N(i, a)$: Q-value for state i and action a at terminal stage (N).
 $g_n(i, a, j)$: Single stage reward for stage n where current state is i , action is a and next state is j .
 $g_N(i)$: Terminal reward at the N^{th} stage when terminal state is i .
 $A(j)$: Set of feasible actions in state j .
 $\eta(i, a)$: Sampling function taking input (i, a) as state-action pair and returns the next state.

Input: Samples of the form $(i$ (current state), a (action), r (reward), j (next state)).

Output: Updated Q-value $Q_n^{m+1}(i, a)$ estimated after m iterations of the algorithm.

Initialization: $Q_n^0(i, a) = 0, \forall(i, a), n = 0, \dots, N - 1$, and $Q_N^0(i, a) = g_N(i), \forall(i, a)$

procedure FINITE HORIZON Q-LEARNING

$$a(m) = \left\lceil \frac{1}{(m+1)/10} \right\rceil$$

$j = \eta(i, a)$ (from samples)

$$Q_n^{m+1}(i, a) = (1 - a(m)) \left(Q_n^m(i, a) \right) + a(m) \times \left(g_n(i, a) + \min_{b \in A(j)} Q_{n+1}^m(j, b) \right), \quad n = 0, 1, \dots, N - 1,$$

$$Q_N^{m+1}(i, a) = g_N(i), \forall(i, a) \text{ tuples.}$$

return $Q_n^{m+1}(i, a)$

end procedure

$$h(x) = \begin{cases} ax & \text{if } x \geq 0 \\ -bx & \text{if } x < 0 \end{cases}$$

The holding cost for the stock is given by $h(x)$ where a is the per-unit storage cost and b is the per-unit backlog cost.

Per-stage cost is $c(X_{t+1}, U_t) = h(X_{t+1}) + pU_t$. Find the optimal inventory control strategy to minimize the expected total cost over a finite horizon.

Parameters: Take P_W as a discrete uniform distribution, $P_W \sim \mathcal{U}\{0, 10\}$.

Reference: Page 14 of Aditya Mahajan Slides on Markov Decision Processes: Sequential decision-making with perfect observation. You can also enrich the problem based on the inventory control problem.

Project 2: Call options

An investor has a *call option* to buy one share of a stock at a fixed price p and has T days to *exercise* this option. For simplicity, assume that the investor makes a decision at the beginning of each day.

The investor may decide not to exercise the option but if he does exercise the option when the stock price is s , he effectively gets $(s - p)$.

Assume that the price of the stock varies with independent increments, *i.e.*, the price on day $t + 1$ is

$$S_{t+1} = S_t + W_t$$

where $\{W_t\}_{t \geq 1}$ is an i.i.d. process.

Your task is to create an agent that decides when to exercise (or not exercise) the call option in the given span of T days. Assume $p \in \mathbb{N}$ and assume W_t to be (discrete) uniformly distributed with endpoints $-\epsilon$ and $+\epsilon$ for some $\epsilon \in \mathbb{N}$. For example, for $\epsilon = 5$, $W_t \sim \mathcal{U}\{-5, 5\}$. You can spice up the problem by relaxing some of these assumptions, using different noise models, working with real data, etc.

References:

- Call Option Wikipedia Page
- Aditya Mahajan Notes on Call Option
- Page 23 of Aditya Mahajan Slides on Markov Decision Processes: Sequential decision-making with perfect observation

Project 3: A machine replacement model

Consider a manufacturing process, where the machine used for manufacturing deteriorates over time. Let $\mathcal{S} = \{0, 1, \dots, n\}$ represent the condition of the machine. The higher the value of s , the worse the condition of the equipment.

A decision maker observes the state of the machine and has two options: continue operating the machine or replace it with a new and identical piece of equipment. Operating the machine in state s costs $h(s)$, where $h(\cdot)$ is a weakly increasing function; replacing the machine costs a constant amount K .

When the machine is operated, it's state deteriorates according to

$$S_{t+1} = \min(S_t + W_t, n)$$

where $\{W_t\}_{t \geq 1}$ is an i.i.d. discrete process.

You can assume $W_t \sim \text{Bernoulli}(p)$ for some $p \in (0, 1)$. Also, take $K \in (h(s_i), h(s_{i+1}))$ for some $s_i \in \mathcal{S}$ and $h(x) = x^2$

Reference: Aditya Mahajan Notes on Machine Replacement

Project 4: A House Selling Example

An individual wants to sell his house and receives an offer at the beginning of each day. Assume that the successive offers are independent and an offer is j with probability P_j , $j = 0, 1, \dots, N$ (here is j is “ranking” of offers). We suppose, however, that any offer not immediately accepted is not lost but may be accepted at any later date. Also, a maintenance cost of C is incurred each day the house remains unsold.

The state at time t will be the largest offer received up to t (including the offer at t). Therefore, (i is currently the best offer “rank”, j is the next day offer “rank”)

$$P_{ij} = \begin{cases} 0 & j < i \\ \sum_{k=0}^i P_k & j = i \\ P_j & j > i \end{cases}$$

Assume $N = 25, i = 10$, and P to follow a uniform distribution. Find the optimal strategy.

Reference: Applied Probability Models with Optimization Application, Sheldon Ross (Chapter 6).

Project 5: Windy Gridworld (Stochastic Wind)

Figure 1 shows a standard grid world, with start and goal states, but with one difference: there is a crosswind running upward through the middle of the grid. Assume eight possible actions: **up**, **down**, **right**, and **left** and also the diagonal moves.

In the middle region, the resultant next states are shifted upward by a “wind”, the strength of which varies from column to column. Assume that the effect of the wind, if there is any, is stochastic, sometimes varying by 1 from the mean value given below each column. That is, a third of the time you move exactly according

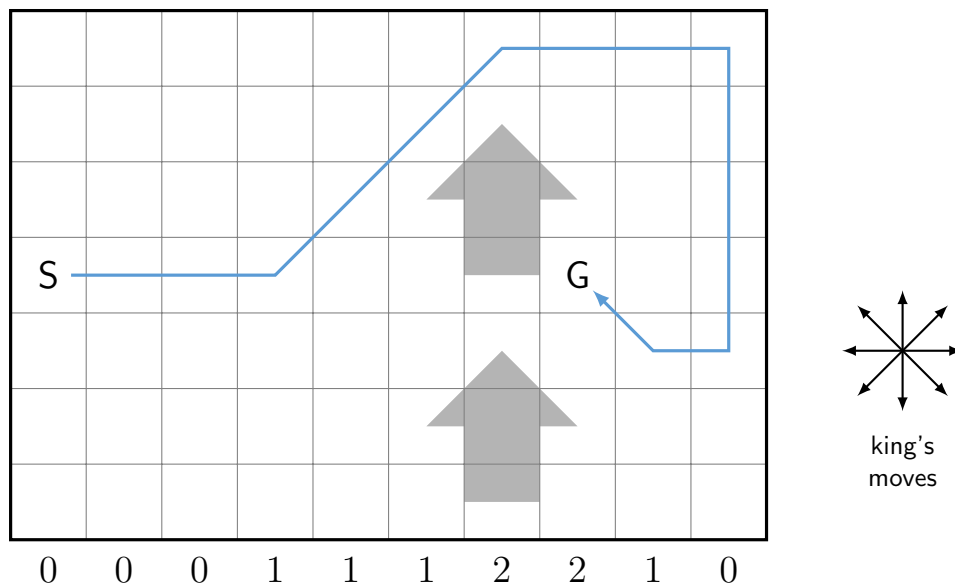


Figure 1: Windy Gridworld

to these values but also a third of the time you move one cell above that, and another third of the time you move one cell below that. For example, if you are one cell to the right of the goal and you move **left**, then one-third of the time you move one cell above the goal, one-third of the time you move two cells above the goal, and one-third of the time you move to the goal. Note there's no wind in the columns that have the value 0 given below them.

Your task is to learn the optimal policy for this setting.

References:

- Pages 130, 131 of Sutton and Barto - Reinforcement Learning: An Introduction (2nd Edition)
- 49:43 timestamp onwards of RL Course by David Silver - Lecture 5: Model Free Control

Project 6: Riverswim

The RiverSwim MDP models an agent swimming in a river who can choose to swim either left or right. The MDP consists of six states arranged in a chain with the agent starting in the leftmost state ($s = 1$). If the agent decides to move left i.e with the river current then it is always successful but if it decides to move right it might fail with some probability. If the final state of the agent is s_1 , the agent receives an award of 0.005. If the final state of the agent is s_6 , the agent receives an award of 1. Each episode is reset every $H = 20$ steps. The MDP state diagram is shown below:

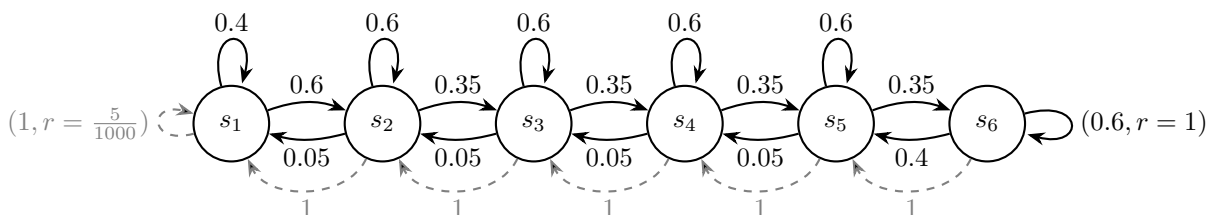


Figure 2: RiverSwim

In the above figure, Solid and dotted arrows denote the transition probabilities for actions ‘right’ and ‘left’, respectively.

Reference: Strehl and Littman - An analysis of model-based interval estimation for Markov decision processes

Project 7: Finite inventory pricing

Assume you operate a chartered plane of K seats with a booking window of T days.i.e., the plane leaves from A to B after every T days. Bookings can be made only for that particular segment and for the next departing flight and not for future departures. As an operator, you want to come up with an optimal pricing policy $p_t, t = 1, 2, \dots, T$ that will maximize your total revenue over T days. Time is discrete, and on day t , the demand for seats at price p_t , denoted by $d(p_t)$ is a random variable supported on positive integers. Some examples are Bernoulli, Poisson, Geometric random variables. Note that if available seats are less than the demand on a day, then some demand is lost (but there is no cost for lost sales). First perform VI or PI to get the optimal pricing policy. Now assume that the demand function is unknown and use RL algorithms to see if the optimal policy is learnt.

Project 8: Thermostat control (discrete)

Consider a thermostat control problem where the goal is to control a heater to keep the indoor temperature close to a desired setpoint while minimizing energy cost and comfort penalty. Let $\mathcal{S} = \{18, 19, 20, 21, 22\}$ represent the discretized indoor temperature in degrees Celsius. A decision maker observes the current temperature $T_t \in \mathcal{S}$ and has two options: keep the heater OFF ($a = 0$) or turn the heater ON ($a = 1$).

When the heater is operated, the continuous indoor temperature evolves according to the stochastic physics approximation:

$$T_{t+1} = T_t + \alpha(T_{out} - T_t) + \beta a + \eta_t$$

where $T_{out} = 10^\circ$ is the fixed outside temperature, and $\eta_t \sim \mathcal{N}(0, \sigma^2)$ is a Gaussian noise variable. To maintain a finite state space, the resulting temperature T_{t+1} is discretized to the nearest integer in $\mathcal{S}$. Assume parameters $\alpha = 0.1$, $\beta = 1.0$, and $\sigma = 0.2$. You can compute the exact transition probabilities $P(s'|s, a)$ by integrating the Gaussian mass that falls into each discrete bin.

At each time step, a cost is incurred based on energy usage and discomfort. The per-step reward is given by $r(s, a, s') = -(c_e a + c_c |T' - T_{set}|)$, where $c_e = 0.5$ is the energy cost per time-step when the heater is ON, $c_c = 1.0$ is the comfort penalty, and $T_{set} = 21^\circ$ is the desired setpoint. Find the optimal policy assuming $H = 50$ time steps, a start state of $T_0 = 19$.

Project 9: Battery charge scheduling

A battery connected to the grid can be charged when electricity prices are low and discharged when prices are high. Assume time is discrete and the goal is to find an optimal charging/discharging policy over a finite horizon of H steps.

Let X_t denote the battery state-of-charge (SoC) *before* the decision at time t , where

$$X_t \in \{0, 1, \dots, B\}.$$

Let P_t denote the (discretized) electricity price at time t , where

$$P_t \in \{0, 1, \dots, N\}.$$

At time t , the operator chooses an action U_t representing the net energy transferred *into* the battery:

$$U_t \in \{-U, -U + 1, \dots, 0, \dots, U - 1, U\}.$$

Here $U_t > 0$ means charging (buying U_t units from the grid), $U_t < 0$ means discharging (selling $|U_t|$ units to the grid), and $U_t = 0$ means idle. The action must be feasible:

$$0 \leq X_t + U_t \leq B.$$

The battery SoC evolves according to

$$X_{t+1} = X_t + U_t.$$

Assume the price follows a bounded random-walk model with independent increments:

$$P_{t+1} = \min\{\max\{P_t + W_t, 0\}, N\},$$

where $\{W_t\}_{t \geq 0}$ is an i.i.d. discrete noise process.

At each time step, the operator incurs a trading cost (negative cost corresponds to profit) and a degradation cost. Define the per-stage cost as

$$c(X_t, U_t, P_t) = P_t U_t + \lambda |U_t|.$$

Note: when $U_t < 0$ (discharge), the term $P_t U_t$ is negative and represents revenue. Optionally, include a terminal penalty for ending the horizon far from a desired SoC X_{ref} :

$$g(X_H) = \rho |X_H - X_{\text{ref}}|.$$

Find the optimal charging/discharging strategy to minimize the expected total cost over a finite horizon:

$$\mathbb{E} \left[\sum_{t=0}^{H-1} c(X_t, U_t, P_t) + g(X_H) \right].$$

Parameters: Take $B = 10$, $U = 2$, $N = 20$, $X_0 = 5$, $P_0 = 10$, $H = 50$. Take $W_t \sim U\{-\varepsilon, \dots, \varepsilon\}$ with $\varepsilon = 2$. Let $\lambda = 0.1$, $X_{\text{ref}} = X_0$, and $\rho = 5$.